

System Skills A02 Report

Waranyu Kittikamet (6780676)

Assignment 2 tasks us with building a finance tracker program utilizing a dynamic linked list structure in C. The following will be an explanation of my finance tracker program. I will be describing how I implemented a linked list structure and its advantages, as well as each source file and how they work.

Linked List Structure

My linked list structure follows a circular linked list structure, by having the “head” of a linked list node connect to the “tail” of the linked list. Essentially, when we are the first node of a linked list and we try to go back, before the head, we end up at the end of the linked list, the tail. Choosing this type of linked list will enable us to more easily traverse the list, making the max walking time half the length of the list, since if we want to access a node that is near the end of the list, we can easily get to it faster by traversing in reverse.

Source Files

linkedlist.h

```
GNU nano 7.2 linkedlist.h
#ifndef LINKEDLIST_H
#define LINKEDLIST_H

// Defining the LinkedList Node structure;
typedef struct Node {
    struct Node *head;
    long amnt;
    char *desc;
    char *status;
    struct Node *tail;
} Node;

// Functions used for LinkedList Nodes
void addlast(Node **list, long value, char *words, int *length, long *total, char *stat);
void add(Node **list, long value, char *words, int pos, int *length, long *total);
void dlt(Node **list, int pos, int *length, long *total);
void clear_memory(Node **list);
void print(Node *list, int *length);

#endif
```

Our node struct is defined as having five variables:

1. **Node *head**: the connecting point of the node for the node that comes before it.
2. **long amnt**: the amount of currency that is being transacted. We choose long to support bigger values.
3. **char *desc**: the description of the payment. We use a char pointer so that the size allocation can happen dynamically
4. **char *status**: the status of the transaction (new, saved, ++ i, --d).
5. **struct Node *tail**: the connecting point of the node for the node that comes after it.

There are also five functions

1. **void addlast**: adds a node to the end of the linked list, as well as updates the total currency and length of the list.
2. **void add**: adds a node to a specific position in the linked list, as well as update the total currency and length of the list.
3. **void dlt**: marks a node at a specific position for deletion, as well as updating the total currency.
4. **void clear_memory**: clears all memory allocation for the list.
5. **void print**: prints all transactions (description, amount, and status).

We use a double pointer (Node **list) so that we can get access to the main pointer (list) so that updating is possible, instead of copying it into the function. The **print** function doesn't need it since all it needs to do is read and print.

llfunc.c

```
void addlast(Node **list, long value, char *words, int *length, long *total, char *stat){
    // Initialize the new transaction.
    Node *n = (Node*)malloc(sizeof(Node));
    if (n == NULL){
        printf("\nNode memory allocation failed\n");
        printf("\nFailed to add transaction\n");
        return;
    }
    n->amnt = value;
    // Allocate space for transaction description. The amount of memory corresponds to how long the
description is.
    n->desc = (char*)malloc(strlen(words) + 1);
    strcpy(n->desc, words);
    // Allocate space for the status of the added transaction.
    n->status = (char*)malloc(strlen(stat) + 1);
    strcpy(n->status, stat);

    if (*list == NULL){
        // In the case of an empty list, initialize the first node by having its head and tail
point to each other.
        n->tail = n;
        n->head = n;
        *list = n;
    }
    else {
        // Get both the first and last nodes.
        Node *first = *list;
        Node *last = first->head;

        // Set the last node's tail and the first node's head pointing to new transaction
        n->tail = first;
        n->head = last;
        last->tail = n;
        first->head = n;
    }
    if (value >= (long)0){
        printf("Income added.");
    }
    else {
        printf("Expense added.");
    }
    // Update the total number of currency after transactions and the length of the list.
    *total += value;
    (*length)++;
}
```

Explanation:

This function creates a new node and attaches it to the end of our circular linked list, essentially updating it. This new node contains the amount, description and status of the transaction as specified in the parameters. In addition, the function also updates the total transaction amount and length of the linked list.

The function first initializes a new Node pointer and allocates space, size of our Node datatype, for it using **malloc**. It then checks if the memory allocation succeeded by confirming if our new Node pointer **n*** is a NULL pointer. If it's not then we go ahead and assign its amount (**amnt**), description (**desc**) and status (**status**). With **desc** and **status** specifically, we use string pointers, therefore necessitating memory allocation. Once that is done, we connect the head

and tail of `n*` either to each other if the list is empty, or to the last and first node respectively. Finally, we inform the user of the income/expense added, and then increment the total transaction amount and length of the linked list.

```
void add(Node **list, long value, char *words, int pos, int *length, long *total){
    // Condition for an out of bounds position.
    if (pos > *length || pos < 0){
        printf("\nPosition is out of bounds\n");
        printf("\nFailed to add transaction\n");
        return;
    }
    // Initialize the new transaction.
    Node *n = (Node*)malloc(sizeof(Node));
    if (n == NULL){
        printf("\nNode memory allocation failed\n");
        printf("\nFailed to add transaction\n");
        return;
    }
    n->amnt = value;
    // Allocate space for the description and status of the transaction
    n->desc = (char*)malloc(strlen(words) + 1);
    strcpy(n->desc, words);
    n->status = (char*)malloc(strlen("+++ i") + 1);
    strcpy(n->status, "+++ i");
    Node *current = *list;

    // Find the node at the designated position.
    if (pos > *length/2){
        current = current->head;
        for (int i = 0; i < *length-pos; i++){
            current = current->head;
        }
    }
    else {
        for (int i = 0; i < pos; i++){
            current = current->tail;
        }
    }

    // Connect the ends.
    Node *prevNode = current->head;
    n->head = prevNode;
    n->tail = current;
    prevNode->tail = n;
    current->head = n;

    if (pos == 0) {
        *list = n;
    }

    if (value >= (long)0){
        printf("Income added at position %d.", pos);
    }
    else {
        printf("Expense added at position %d.", pos);
    }
    // Update the total number of currency after transactions and the length of the list.
```

```

    *total += value;
    (*length)++;
}

```

Explanation:

This function creates a new node and attaches it to the user specified position in the linked list. All the processes used in **addlast** are also used here. There are two exceptions. This function checks if the given position is out of the bounds of the list (less than 0 or more than the current length of the linked list). The Node traversal is also different. If the given position is more than half the length of the list, then list traversal will go in reverse (tail to head). Otherwise, go forward (head to tail).

```

void dlt(Node **list, int pos, int *length, long *total){
    // Check if the given position is out of bounds of the length of the list.
    if (pos >= *length || pos < 0){
        printf("\nPosition is out of bounds\n");
        printf("\nFailed to mark transaction for deletion\n");
        return;
    }
    if (*list == NULL){
        printf("\nNo transactions exists to delete\n");
        printf("\nFailed to mark transaction for deletion\n");
        return;
    }
    Node *current = *list;
    /*
        For a position that is past half way the length of the list, then we will traverse the
list from the tail to the head
        Otherwise, traverse normally from the head to the tail
    */
    if (pos > *length/2){
        for (int i = 0; i < *length-pos; i++){
            current = current->head;
        }
    }
    else {
        for (int i = 0; i < pos; i++){
            current = current->tail;
        }
    }

    free(current->status);
    current->status = (char*)malloc(strlen("--- d") + 1);
    strcpy(current->status, "--- d");
    *total -= current->amnt;
    printf("Transaction at position %d marked for deletion.", pos);
    return;
}

```

Explanation:

This function marks a user specified transaction for deletion using its position. The Node traversal and out-of-bounds check are all lifted from **add**. Once the function reaches the Node, it changes its status to “--- d” to mark it for deletion.

```

void clear_memory(Node **list){
    if (*list == NULL || list == NULL){ return; }
}

```

```

Node *current = *list;
Node *nextNode;
current->head->tail = NULL;

while (current != NULL){
    nextNode = current->tail;
    free(current->desc);
    free(current->status);
    free(current);
    current = nextNode;
}

*list = NULL;
printf("\nMemory cleared\n");
}

```

Explanation:

This function frees all the memory allocated for the list, its nodes, and their respective information. It does so by going Node by Node, each time freeing the memory in the current node before freeing the node itself and going to the next node. This is repeated until all nodes are freed, then the list gets turned into a NULL pointer.

```

void print(Node *list, int *length){
    if (list == NULL){
        printf("[ Transactions ]\n(Empty)\n");
        return;
    }
    Node *current = list;
    printf("[ Transactions ]\n");
    for (int i = 0; i < *length; i++){
        printf("%d. %s\t%d\t%s\n", i+1, current->desc, current->amnt, current->status);
        current = current->tail;
    }
}

```

Explanation:

This function prints the description, amount and status of all the transactions. It does so in the following format:

```

[ Transactions ]
1. [desc] [amnt] [status]
2. [desc] [amnt] [status]
3. [desc] [amnt] [status]

```

main.c

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#include "../include/linkedlist.h"

int main(){
    // ---Variable Definition---
    char conf[200]; // conf will be used to read the user input
    long t = 0; // t is the total amount of currency left after all transactions (positive amount
being within budget, negative amount being over budger)
    long *total = &t; // *total will be the pointer to t. We will be using a pointer in our
functions so that we can update t.

```

```

int l = 0; // l is the length of the circular, dynamic linked list.
int *length = &l; // *length will be the pointer to l. Same purpose as *total
char desc[100]; // desc will hold a description for a transaction until stored in a linked list
node.
long amnt; // amnt will hold the transaction amount until stored in a linked list node.
char status[10]; // status will hold the status of the transaction until stored in a linked list
node.
Node *list = NULL; // *list will be are linked list. Initialize as a NULL pointer until we
finish some pre-requisite checks

// ---Intro Text---
printf("**Starting the Program (Loading Previous Transactions)**\n");
printf("\n**Original Log File (logs/transaction_log.txt) Before Running the Program:**\n");
printf("```\n# Format: TYPE|DESCRIPTION|AMOUNT\n# TYPE: INC (Income) or EXP (Expense)\n# AMOUNT:
Positive decimal number\n");
printf("\nINC|Freelance Project|250.00\nEXP|Phone Bill|-45.00\nEXP|Electricity
Bill|-100.00\n```\n");
printf("\n**Program Startup Output:**\n");
printf("\n```\nWelcome to your Personal Finance Tracker!\n");

```

Explanation:

These are some of the variables we will need to initialize at the very start of the program running. This is because of processes, like pending user input and reloading a previous session, that go before the main process of the fintrack program (adding/deleting income/expenses).

```

// Section 1: Asking for user input (reloading previous session)
while(1) {
    printf("\nWould you like to resume your previous session? (y/n): ");
    // Read the line of text from input(stdin)
    if (fgets(conf, sizeof(conf), stdin) != NULL) {
        // Replace the newline with a null terminator for the if-conditions
        conf[strcspn(conf, "\n")] = '\0';
        if (strcmp(conf, "y") == 0 || strcmp(conf, "n") == 0) {
            break;
        }
        else {
            printf("\nConfirmation not recognized\n");
        }
    }
}

```

Explanation:

This section is responsible for asking the user whether they want to reload a previous session or not. The while loop using **while(1)** will keep whatever is running in the loop running infinitely, since the looping condition is constant, thus keeping pending the user for a confirmation. **fgets(conf, sizeof(conf), stdin)** then takes the user input (**stdin**) of the same size as the variable **conf**. If **conf** is not NULL, then we will proceed to check if the input is either “y” or “n”. In order to do so, we need to replace the newline in the user input with a NULL terminator so that we can properly compare it to another string. We use **strcspn(conf, "\n")** to find the position of the newline, then we pass it to **conf[strcspn(conf, "\n")] = '\0'**; to replace the newline. After that we use **strcmp** to compare the user input to our allowed terms. If they match, then we exit the loop.

```

// Section 2: Reloading previous transaction_log.txt
/*
    Insert all contents from the transaction_log.txt into the list
    If transaction_log.txt is empty or doesn't exist, start with an empty list.

```

```

*/
if (strcmp(conf, "y") == 0){
    printf("\nResuming from last session...\n\n");
    FILE *log = fopen("../logs/transaction_log.txt", "r");
    // Check if transaction_log.txt exists in the logs folder
    if (log != NULL){
        int empt_check = 0; // int value that will increment by one every loop. If this
and num of arguments is 0, transaction_log.txt
        // conf is used to store the user inputted line
        while (fgets(conf, sizeof(conf), log) != NULL) {
            int p = sscanf(conf, "%s\t%ld\t%s", desc, &amnt, status);
            // Check if transaction_log.txt is empty by looking at the first line.
            if (empt_check == 0 && p != 3){
                printf("No previous session detected. Initiating new
session.\n");
                break;
            }

            // If the transaction_log.txt has an entry that does not follow our
saving pattern, then we exit the program
            // Saving pattern: desc[tab space]amnt[tab space]status
            if (p != 3){
                printf("Previous session is corrupted");
                clear_memory(&list);
                return 1;
            }
            status[strcspn(status, "\n")] = '\0';
            addlast(&list, amnt, desc, length, total, status);
            printf("\n");
        }
        printf("\nLast session loaded.\n");
        fclose(log);
    }
    else {
        printf("\nNo previous transactions found. Continuing with new transaction.\n");
        fclose(log);
    }
}

```

Explanation:

This section is responsible for resuming a previous session of fintrack if the user requests for it. If we get a “y” from the previous section, then we proceed to open transaction_log.txt in read mode using **FILE *log = fopen("../logs/transaction_log.txt", "r");**. If the file does not exist or is empty, we will proceed anyways to the next section. If the file does exist, then we read the contents of transaction_log.txt line by line, and add the contents each to our linked list using **addlast** to preserve the order in transaction_log.txt. If transaction_log.txt has anything in it that is not in the format we want, then we inform the user that their previous session is corrupted and close the program. We use **fgets(conf, sizeof(conf), log)** to get each line and **int p = sscanf(conf, "%s\t%ld\t%s", desc, &amnt, status);**. If one of the entries does not follow our format ($p \neq 3$, p tells us how many arguments follow the correct format and we are asking for three arguments), then we exit the program. Otherwise, we replace the newline in status, since it is the last argument in each line in transaction_log.txt, and then we add all the saved information into the linked list. Repeat until every item in transaction_log.txt has been loaded. Finally we close transaction_log.txt using **close(log)**.

```

// Section 3: Transaction Management
printf("\nAvailable actions:\nadd income [amount] [description]\nadd income [amount]
[description] [position]\n");

```

```

printf("add expense [amount] [description]
[position]\ndelete [position]\nprint\nquit\n");
printf("Disclaimer: [position] starts from 1\n");
while(1) {
    int wscount=0; // wscount is our counter for whitespace in the user input
    char cmd[10]; // cmd will represent the first argument given in the user input
    char cmd2[10]; // cmd2 will represent the second argument given in the user input
    int pos; // pos will hold the user inputted position
    int p; // p will be used for sscanf and tell us how many arguments in the user input

    // match the specified pattern
    printf("\nCurrent total: %ld\n", *total);
    if (*total >= (long)0){
        printf("Budget Status: Within Budget\n");
    }
    else {
        printf("Budget Status: Over Budget!\n");
    }
    printf("```\n\n`");
    // This part takes the user inputted line of arguments
    printf("\n> ");
    if (fgets(conf, sizeof(conf), stdin) != NULL) {
        // Replace the newline with a NULL terminator so the string can be used in
        // comparisons
        conf[strcspn(conf, "\n")] = '\0';
        if (strcmp(conf, "quit") == 0) {
            break;
        }
        // Counting the whitespace between arguments helps indicate what command the
        // user inputted
        for(int i = 0; i < strlen(conf); i++){
            if (conf[i] == ' '){ wscount++; }
            if (wscount == 4){ break; }
        }
        switch(wscount){
            // if command == "add [income/expense] [amount] [desc] [pos]" ->
            // add(&list, [amnt/-amnt], desc, pos-1, length, total);
            // Four whitespaces indicate adding an income/expense at a certain
            // position
            case 4:
                p = sscanf(conf, "%s %s %ld %s %d", cmd, cmd2, &amnt, desc,
                &pos);

                if (p == 5 && strcmp(cmd, "add") == 0){
                    // Negative numbers are not allowed to be inputted as a
                    // transaction
                    if (amnt < (long)0){
                        printf("\nInvalid transaction amount\n");
                    }
                    else if (strcmp(cmd2, "income") == 0){
                        add(&list, amnt, desc, pos-1, length, total);
                        printf("\n");
                    }
                    else if (strcmp(cmd2, "expense") == 0){
                        add(&list, -amnt, desc, pos-1, length, total);
                        printf("\n");
                    }
                    else {
                        printf("\nInvalid Command\n");
                    }
                }
            }
        }
    }
}

```



```

    }
    }
    else {
        printf("\nInvalid Command\n");
    }
    break;
    // if command == "add [income/expense] [amount] [desc]" ->
addlast(&list, [amnt/-amnt], desc, length, total, "(new)");
    // Three whitespaces indicate adding an income/expense with no location
given

    case 3:
        p = sscanf(conf, "%s %s %ld %s", cmd, cmd2, &amnt, desc);
        if (p == 4 && strcmp(cmd, "add") == 0){
            if (amnt < (long)0){
                printf("\nInvalid transaction amount\n");
            }
            else if (strcmp(cmd2, "income") == 0){
                addlast(&list, amnt, desc, length, total,
"(new)");

                printf("\n");
            }
            else if (strcmp(cmd2, "expense") == 0){
                addlast(&list, -amnt, desc, length, total,
"(new)");

                printf("\n");
            }
            else {
                printf("\nInvalid Command\n");
            }
        }
        else {
            printf("\nInvalid Command\n");
        }
        break;
    // if command == "delete [pos]" -> dlt(&list, pos-1, length, total);
    // One whitespace indicate deleting a transaction at a certain position
    case 1:
        p = sscanf(conf, "%s %d", cmd, &pos);
        if (p == 2 && strcmp(cmd, "delete") == 0){
            dlt(&list, pos-1, length, total);
            printf("\n");
        }
        else{
            printf("\nInvalid Command\n");
        }
        break;
    // if command == "print" -> print(list, length);
    // No whitespace indicates print
    default:
        if (strcmp(conf, "print") == 0){
            print(list, length);
        }
        else {
            printf("\nInvalid Command\n");
        }
        break;
}

```

```

    }
}

```

Explanation:

This section handles the process of adding/deleting incomes/expenses and/or print the current list of transactions. First we use a **while(1)** loop to keep pending for user input, then we get the user input using **fgets(conf, sizeof(conf), stdin)**. We then clean up the user input using **conf[strcspn(conf, "\n")] = '\0'**; and then we count the whitespaces in the user input. The amount of whitespaces can clue us into what command the user wants to input. For example, "add income 300 gift 3" has 4 whitespaces in between the arguments, telling us that it's an add command specifying a position. We use **switch(wscount)** to indicate and direct all our whitespace cases. Case 4 is for adding an income/expense at a specific position. Case 3 is for adding an income/expense at the end of the list. Case 1 is for deleting a transaction at a specific position. And default is for print. Each case has conditions for handling incorrect/unrecognized commands including checking **p** for the correct arguments and using **strcmp** to compare **cmd2** to "income"/"expense". The addition/insertion commands also have an additional condition where if the inputted amount is negative, it will not add/insert the transaction. Finally, the user can type "quit" to exit this section and proceed with saving the current transactions.

```

// Section 4: Saving and Clearing
// Save all transactions into ../logs/transaction_log.txt, clear memory and exit the program.
printf("\nSaving transactions to file...\n\n");
FILE *log = fopen("../logs/transaction_log.txt", "w");
if (log == NULL){
    printf("\nUnable to save\n");
    clear_memory(&list);
    return 1;
}
if (list != NULL) {
    printf("# Format: TYPE|DESCRIPTION|AMOUNT\n# TYPE: INC (Income) or EXP (Expense)\n#
AMOUNT: Positive decimal number\n\n");
    Node *current = list;
    for (int i = 0; i < *length; i++){
        // Indicate the ones that are marked for deletion, and then continue to the next
        Node
        if (strcmp(current->status, "--- d") == 0){
            printf("- **%s (%ld) was removed**\n", current->desc, current->amnt);
            current = current->tail;
            continue;
        }
        // Each transaction gets saved into transaction_log.txt in the following format:
[desc] [amnt] (saved)
        else {
            fprintf(log, "%s\t%ld\t(saved)\n", current->desc, current->amnt);
            // Indicate the ones that were inserted at a specific position
            if (strcmp(current->status, "+++ i") == 0){
                if (current->amnt >= (long)0){
                    printf("INC(insert)|%s|%ld\n", current->desc,
current->amnt);
                }
                else {
                    printf("EXP(insert)|%s|%ld\n", current->desc,
current->amnt);
                }
            }
            else if (current->amnt >= (long)0){

```

```

                                printf("INC%s|s|%ld\n", current->status, current->desc,
current->amnt);
                                }
                                else {
                                printf("EXP%s|s|%ld\n", current->status, current->desc,
current->amnt);
                                }
                                }
                                current = current->tail;
                        }
                }
                fclose(log);
                clear_memory(&list);
                printf("\nDone. Exiting program.\n");
                return 0;
}

```

Explanation:

This section is responsible for saving the current session of transactions and also clearing memory that was allocated for the linked list. We first open `transaction_log.txt` in write mode using **FILE *log = fopen("../logs/transaction_log.txt", "w");** and then check if the file exists. If it doesn't, then we just clear the memory and exit the program. If the file does exist, then we begin to overwrite. If a transaction has been marked for deletion ("--- d"), then we skip to the next transaction. Otherwise, we use **fprintf(log, "%s\t%ld\t(saved)\n", current->desc, current->amnt);** to print our transaction into `transaction_log.txt` in our desired format: desc[tab space]amnt[tab space]status. We do this to every transaction in our current session, with every transaction getting the status "(saved)", and then after all transactions have been saved, we clear the memory allocated for the linked list and quit the program.

Program Demonstration (Screenshots)

```
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2$ ls -a
.  ..  .git  Makefile  README.md  dist  include  logs  release  report  src
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2$ cd logs
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2/logs$ cat transaction_log.txt

trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2/logs$ cd ..
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2$ cd release
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2/release$ ./fintrack
**Starting the Program (Loading Previous Transactions)**

**Original Log File (logs/transaction_log.txt) Before Running the Program:**
...
# Format: TYPE|DESCRIPTION|AMOUNT
# TYPE: INC (Income) or EXP (Expense)
# AMOUNT: Positive decimal number

INC|Freelance Project|250.00
EXP|Phone Bill|-45.00
EXP|Electricity Bill|-100.00
...

**Program Startup Output:**
...
Welcome to your Personal Finance Tracker!

Would you like to resume your previous session? (y/n): y

Resuming from last session...

No previous session detected. Initiating new session.

Loading completed.

Available actions:
add income [amount] [description]
add income [amount] [description] [position]
add expense [amount] [description]
add expense [amount] [description] [position]
delete [position]
print
quit
Disclaimer: [position] starts from 1

Current total: 0
Budget Status: Within Budget
```

```
'''

'''

> print
[ Transactions ]
(Empty)

Current total: 0
Budget Status: Within Budget
'''

'''

> add income 300 Allowance
Income added.

Current total: 300
Budget Status: Within Budget
'''

'''

> add income 170 Gift
Income added.

Current total: 470
Budget Status: Within Budget
'''

'''

> add expense 100 Bill
Expense added.

Current total: 370
Budget Status: Within Budget
'''

'''

> add expense 45 Lunch
Expense added.

Current total: 325
Budget Status: Within Budget
'''

'''
```

```
> print
[ Transactions ]
1. Allowance      300      (new)
2. Gift 170       (new)
3. Bill -100      (new)
4. Lunch          -45      (new)

Current total: 325
Budget Status: Within Budget
```

...


```
> delete 3
Transaction at position 3 marked for deletion.

Current total: 425
Budget Status: Within Budget
```

...


```
> add expense 500 Renovation
Expense added.

Current total: -75
Budget Status: Over Budget!
```

...


```
> delete 5
Transaction at position 5 marked for deletion.

Current total: 425
Budget Status: Within Budget
```

...


```
> add expense 500 Renovation 3
Expense added at position 3.

Current total: -75
Budget Status: Over Budget!
```

...


```
> add income 150 Borrowed 4
Income added at position 4.
```


```


```


```


```


```

Current total: 75  
Budget Status: Within Budget  
\\

...

```
> print
[Transactions]
1. Allowance 300 (new)
2. Gift 170 (new)
3. Renovation -500 +++ i
4. Borrowed 150 +++ i
5. Bill -100 --- d
6. Lunch -45 (new)
7. Renovation -500 --- d
```

Current total: 75  
Budget Status: Within Budget  
\\

...

```
> add income -500 Lost
```

Invalid transaction amount

Current total: 75  
Budget Status: Within Budget  
\\

...

```
> add expense -1000 Stolen
```

Invalid transaction amount

Current total: 75  
Budget Status: Within Budget  
\\

...

```
> add income Twenty Found
```

Invalid Command

Current total: 75  
Budget Status: Within Budget  
\\

```

> add expense One Hundred and Forty Five Salary

Invalid Command

Current total: 75

Budget Status: Within Budget

```

```

> add income 145 Salary 9

Position is out of bounds

Failed to add transaction

Current total: 75

Budget Status: Within Budget

```

```

> quit

Saving transactions to file...

Format: TYPE|DESCRIPTION|AMOUNT

TYPE: INC (Income) or EXP (Expense)

AMOUNT: Positive decimal number

INC(new)|Allowance|300

INC(new)|Gift|170

EXP(insert)|Renovation|-500

INC(insert)|Borrowed|150

- **Bill (-100) was removed**

EXP(new)|Lunch|-45

- **Renovation (-500) was removed**

Memory cleared

Done. Exiting program.


```
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2/release$ cd ..
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2$ cd logs
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2/logs$ cat transaction_log.txt
Allowance      300      (saved)
Gift      170      (saved)
Renovation    -500      (saved)
Borrowed     150      (saved)
Lunch     -45      (saved)
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2/logs$ cd ..
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2$ cd release
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2/release$ ./fintrack
**Starting the Program (Loading Previous Transactions)**

**Original Log File (logs/transaction_log.txt) Before Running the Program:**
```
Format: TYPE|DESCRIPTION|AMOUNT
TYPE: INC (Income) or EXP (Expense)
AMOUNT: Positive decimal number

INC|Freelance Project|250.00
EXP|Phone Bill|-45.00
EXP|Electricity Bill|-100.00
```

**Program Startup Output:**

```
Welcome to your Personal Finance Tracker!

Would you like to resume your previous session? (y/n): y

Resuming from last session...

Income added.
Income added.
Expense added.
Income added.
Expense added.

Loading completed.

Available actions:
add income [amount] [description]
add income [amount] [description] [position]
add expense [amount] [description]
add expense [amount] [description] [position]
```

```

delete [position]
print
quit
Disclaimer: [position] starts from 1

Current total: 75
Budget Status: Within Budget
'''

...

> print
[Transactions]
1. Allowance 300 (saved)
2. Gift 170 (saved)
3. Renovation -500 (saved)
4. Borrowed 150 (saved)
5. Lunch -45 (saved)

Current total: 75
Budget Status: Within Budget
'''

...

> |

```

## Command Reference Table

| Command / tool                         | What it does                                                                                                          | Key flag(s) you used                                                                                                                                                                                         | Line # in code      | Sample output from your machine |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|---------------------------------|
| Node *n = (Node*)malloc(sizeof(Node)); | Allocates space to Node pointer *n. The amount of space allocated is the same amount as the size of the Node datatype | <p><b>*</b></p> <p>Turns a variable into a pointer.</p> <p><b>(Node*)</b></p> <p>Typecasts the allocated memory as a datatype Node.</p> <p><b>sizeof(Node)</b></p> <p>Returns the size of datatype Node.</p> | Line 14 in llfunc.c | -                               |

|                                              |                                                                                                                                                                                      |                                                                                                                                                                                                                                                           |                     |                               |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|-------------------------------|
| printf("\nNode memory allocation failed\n"); | Prints out "Node memory allocation failed", starting a new line before and after printing the string.                                                                                | \n<br>Print on a new line                                                                                                                                                                                                                                 | Line 15 in llfunc.c | Node memory allocation failed |
| n->desc = (char*)malloc(strlen(words) + 1);  | Allocates space to the <b>desc</b> parameter in Node <b>n</b> . <b>n-&gt;desc</b> is a string and the amount of space allocated is equal to the length of string <b>words</b> + 1    | <b>n-&gt;desc</b><br>String parameter for Node <b>n</b> used for the description of the transaction.<br><br><b>(char*)</b><br>Typecasts the allocated memory as a datatype char.<br><br><b>strlen(words)</b><br>Returns the length of string <b>words</b> | Line 22 in llfunc.c | -                             |
| strcpy(n->desc, words);                      | Copies the string <b>words</b> into string <b>n-&gt;desc</b>                                                                                                                         | <b>n-&gt;desc</b><br>String parameter for Node <b>n</b> used for the description of the transaction.<br><br><b>words</b><br>The description of the transaction.                                                                                           | Line 23 in llfunc.c | -                             |
| n->status = (char*)malloc(strlen(stat) + 1); | Allocates space to the <b>status</b> parameter in Node <b>n</b> . <b>n-&gt;status</b> is a string and the amount of space allocated is equal to the length of string <b>stat</b> + 1 | <b>n-&gt;status</b><br>String parameter for Node <b>n</b> used for the status of the transaction.<br><br><b>(char*)</b><br>Typecasts the allocated memory as a datatype char.<br><br><b>strlen(words)</b><br>Returns the length of string <b>words</b>    | Line 25 in llfunc.c | -                             |
| strcpy(n->status, stat);                     | Copies the string <b>stat</b> into string <b>n-&gt;status</b>                                                                                                                        | <b>n-&gt;status</b><br>String parameter for Node <b>n</b> used for the status of the transaction.<br><br><b>stat</b><br>The status of the transaction.                                                                                                    | Line 26 in llfunc.c | -                             |

|                                                                                             |                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                               |                      |                          |
|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|--------------------------|
| n->status =<br>(char*)malloc(strlen("+++ i") + 1);                                          | Allocates space to the <b>status</b> parameter in Node <b>n</b> . <b>n-&gt;status</b> is a string and the amount of space allocated is equal to the length of string "+++ i" + 1 | <b>n-&gt;status</b><br>String parameter for Node <b>n</b> used for the status of the transaction.<br><br><b>(char*)</b><br>Typecasts the allocated memory as a datatype char.<br><br><b>strlen("+++ i") + 1</b><br>Returns the length of string "+++ i" + 1                                   | Line 79 in llfunc.c  | -                        |
| strcpy(n->status, "+++ i");                                                                 | Copies the string "+++ i" into string <b>n-&gt;status</b>                                                                                                                        | <b>n-&gt;status</b><br>String parameter for Node <b>n</b> used for the status of the transaction.<br><br>"+++ i"<br>The status given to transactions that were added to a specific position.                                                                                                  | Line 80 in llfunc.c  | -                        |
| *total -=<br>current->amnt;                                                                 | Increments the long value pointed at by long pointer <b>*total</b> by long value <b>current-&gt;amnt</b>                                                                         | <b>current-&gt;amnt</b><br>Long parameter for Node <b>current</b> . This is used to access the current node's transaction amount.                                                                                                                                                             | Line 154 in llfunc.c | -                        |
| printf("%d.<br>%s\t%d\t%s\n", i+1,<br>current->desc,<br>current->amnt,<br>current->status); | Prints out a transaction entry (Description, amount and status).                                                                                                                 | <b>%d</b><br>Format specifier for digit.<br><br><b>%s</b><br>Format specifier for string.<br><br><b>%ld</b><br>Format specifier for long digit.<br><br><b>\t</b><br>Tab space.<br><br><b>current-&gt;desc</b><br>String parameter for the current Node, used to access the description of the | Line 198 in llfunc.c | 1. Allowance 300 (saved) |

|                        |                                                                               |                                                                                                                                                                                                                                                                                                                                            |                      |   |
|------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|---|
|                        |                                                                               | <p>currently selected node's transaction.</p> <p><b>current-&gt;amnt</b><br/>Long parameter for Node <b>current</b>. This is used to access the current node's transaction amount.</p> <p><b>current-&gt;status</b><br/>String parameter for the current Node, used to access the status of the currently selected node's transaction.</p> |                      |   |
| *total += value;       | Increases long pointer <b>*total</b> by long value <b>value</b>               | *<br>Turns a variable into a pointer.                                                                                                                                                                                                                                                                                                      | Line 52 in llfunc.c  | - |
| (*length)++;           | Increments integer pointer <b>*length</b> by 1                                | *<br>Turns a variable into a pointer.                                                                                                                                                                                                                                                                                                      | Line 53 in llfunc.c  | - |
| free(current->status); | Frees memory allocated to string <b>status</b> located in Node <b>current</b> | <b>current-&gt;status</b><br>String parameter for the current Node, used to access the status of the currently selected node's transaction.                                                                                                                                                                                                | Line 151 in llfunc.c | - |
| char conf[200];        | Initializes string <b>conf</b> of length 200                                  | -                                                                                                                                                                                                                                                                                                                                          | Line 9 in main.c     | - |
| long t = 0;            | Initializes long <b>t</b> as 0                                                | -                                                                                                                                                                                                                                                                                                                                          | Line 10 in main.c    | - |
| long *total = &t;      | Initializes long pointer <b>*total</b> to address of <b>t</b>                 | *<br>Turns a variable into a pointer.<br><br>&<br>Gets the address of the variable                                                                                                                                                                                                                                                         | Line 11 in main.c    | - |
| Node *list = NULL;     | Initializes a NULL pointer <b>*list</b> with datatype Node                    | *<br>Turns a variable into a pointer.                                                                                                                                                                                                                                                                                                      | Line 17 in main.c    | - |
| while(1) { }           | Infinite while loop                                                           | -                                                                                                                                                                                                                                                                                                                                          | Line 28 in main.c    | - |

|                                                           |                                                                                          |                                                                                                                                                                                                                            |                   |                            |
|-----------------------------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|----------------------------|
| fgets(conf, sizeof(conf), stdin)                          | Assigns user input string from <b>stdin</b> to string variable current->amnt <b>conf</b> | <b>conf</b><br>User input string.<br><br><b>sizeof(conf)</b><br>Length of the user's input string.<br><br><b>stdin</b><br>Gets user's input.                                                                               | Line 31 in main.c | add income 145<br>Salary 9 |
| strcspn(conf, "\n")                                       | Gives the position of the first occurrence of “/n” in string <b>conf</b>                 | <b>conf</b><br>User input string.<br><br><b>"\n"</b><br>Newline character.                                                                                                                                                 | Line 33 in main.c | 12                         |
| conf[strcspn(conf, "\n")] = '\0';                         | Replaces the newline in string <b>conf</b> with a NULL terminator.                       | <b>conf</b><br>User input string.<br><br><b>"\n"</b><br>Newline character.<br><br><b>'\0'</b><br>Null terminator.<br><br><b>strcspn(conf, "\n")</b><br>Returns the index position of string <b>conf</b> that contains “\n” | Line 33 in main.c | -                          |
| strcmp(conf, "y")                                         | Compares strings <b>conf</b> and “y” and gives boolean output based on similarity.       | <b>conf</b><br>User input string.<br><br><b>"y"</b><br>y for confirming “yes”                                                                                                                                              | Line 34 in main.c | 0                          |
| FILE *log = fopen("../logs/transaction_log.txt", "r");    | Initializes a file pointer <b>*log</b> by opening transaction_log.txt in read mode.      | <b>*log</b><br>File pointer for transaction_log.txt<br><br><b>"../logs/transaction_log.txt"</b><br>File path to transaction_log.txt<br><br><b>"r"</b><br>Used with <b>fopen</b> to open a file in read mode.               | Line 50 in main.c | -                          |
| int p = sscanf(conf, "%s\t%ld\t%s", desc, &amnt, status); | Scans string <b>conf</b> to check if it follows the pattern                              | <b>conf</b><br>User input string.                                                                                                                                                                                          | Line 56 in main.c | -                          |

|                                                        |                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                             |                    |                   |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|-------------------|
|                                                        | <p><code>"%s\t%d\t%s"</code>, taking each format specifier and assigning them to their respective variables.</p> | <p><b>%s</b><br/>Format specifier for string.</p> <p><b>\t</b><br/>Tab space.</p> <p><b>%ld</b><br/>Format specifier for long digit.</p> <p><b>%s</b><br/>Format specifier for string.</p> <p><b>desc</b><br/>Description of transaction inputted by the user.</p> <p><b>&amp;amnt</b><br/>Transaction amount inputted by the user.</p> <p><b>status</b><br/>Status of the transaction.</p> |                    |                   |
| <code>fclose(log);</code>                              | Closes file pointer <b>log</b> .                                                                                 | <p><b>log</b><br/>File pointer to transaction_log.txt</p>                                                                                                                                                                                                                                                                                                                                   | Line 75 in main.c  | -                 |
| <code>printf("\nCurrent total: %ld\n", *total);</code> | Prints out the current total of our transactions using long pointer <b>*total</b> .                              | <p><b>\n</b><br/>Newline character.</p> <p><b>%ld</b><br/>Format specifier for long digit.</p> <p><b>*total</b><br/>Total transaction amount.</p>                                                                                                                                                                                                                                           | Line 93 in main.c  | Current total: 75 |
| <code>strlen(conf);</code>                             | Gives the length of string <b>conf</b>                                                                           | <p><b>conf</b><br/>User input string.</p>                                                                                                                                                                                                                                                                                                                                                   | Line 110 in main.c | 28                |
| <code>switch(wscount) { }</code>                       | Switch case function                                                                                             | <p><b>wscount</b><br/>Number of whitespaces in user input line.</p>                                                                                                                                                                                                                                                                                                                         | Line 114 in main.c | -                 |
| <code>p = sscanf(conf, "%s %s %ld %s %d",</code>       | Scans string <b>conf</b> to check if it follows the pattern <code>"%s %s %ld</code>                              | <p><b>%s</b><br/>Format specifier for string.</p>                                                                                                                                                                                                                                                                                                                                           | Line 118 in main.c | -                 |

|                                                                                             |                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                           |                               |
|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|-------------------------------|
| <pre>cmdnd, cmdnd2, &amp;amnt, desc, &amp;pos);</pre>                                       | <p>%s %d", taking each format specifier and assigning them to their respective variables.</p> | <p><b>%ld</b><br/>Format specifier for long digit.</p> <p><b>%d</b><br/>Format specifier for digit.</p> <p><b>cmdnd</b><br/>First argument in the user input line.</p> <p><b>cmdnd2</b><br/>Second argument in the user input line.</p> <p><b>&amp;amnt</b><br/>Transaction amount inputted by the user.</p> <p><b>desc</b><br/>Description of transaction inputted by the user.</p> <p><b>&amp;pos</b><br/>Position in the linked list inputted by the user.</p> |                           |                               |
| <pre>FILE *log = fopen("../logs/transac tion_log.txt", "w");</pre>                          | <p>Initializes a file pointer <b>*log</b> by opening transaction_log.txt in write mode.</p>   | <p><b>*log</b><br/>File pointer for transaction_log.txt</p> <p><b>"../logs/transaction_log.txt"</b><br/>File path to transaction_log.txt</p> <p><b>"w"</b><br/>Used with <b>fopen</b> to open a file in write mode.</p>                                                                                                                                                                                                                                           | <p>Line 193 in main.c</p> | -                             |
| <pre>printf("INC%s s l d\n", current-&gt;status, current-&gt;desc, current-&gt;amnt);</pre> | <p>Prints out a saved income transaction.</p>                                                 | <p><b>%s</b><br/>Format specifier for string.</p> <p><b>%ld</b><br/>Format specifier for long digit.</p>                                                                                                                                                                                                                                                                                                                                                          | <p>Line 222 in main.c</p> | <p>INC(new) Allowance 300</p> |



|  |  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |  |  |
|--|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
|  |  | <p><b>\n</b><br/>Newline character.</p> <p><b>current-&gt;status</b><br/>String parameter for the current Node, used to access the status of the currently selected node's transaction.</p> <p><b>current-&gt;desc</b><br/>String parameter for the current Node, used to access the description of the currently selected node's transaction.</p> <p><b>current-&gt;amnt</b><br/>Long parameter for Node <b>current</b>. This is used to access the current node's transaction amount.</p> |  |  |
|--|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|